Predictive Deadlock Warning — the paper explained simply

Every idea, table and figure, and what to conclude from each

Revanth Reddy Lingala and Padigela Srujan

Contents

1. The paper in one minute **2**

2. Background you need **2**
2.1 Resources, claims and needs . . . 2
2.2 Safe, unsafe and deadlocked . . . 3
2.3 Why "observer mode"? . . . 3

3. What we built **3**
3.1 The simulator . . . 3
3.2 Three workloads (kinds of behaviour) . . . 3
3.3 The five warning signals (indicators) . . . 3
3.4 Our method: the rule-based score . . . 4
3.5 The twelve methods we compared . . . 4
3.6 How we judged a warning . . . 5
3.7 Fair testing . . . 5
3.8 Two pieces of statistics, explained . . . 5

4. The three hypotheses and their verdicts **5**

5. Every table explained **6**
Table I — Scope of related work . . . 6
Table II — Methods compared . . . 6
Table III — Claim taxonomy ("what we claim and what we don't") . . . 6
Table IV — Experimental runs . . . 6
Table V — Test set per workload and size . . . 7
Table VI — Main results (the most important table) . . . 7
Table VII — Paired differences across 20 batches . . . 7
Table VIII — All methods on the common subset . . . 8
Table IX — Ablation (removing one signal at a time) . . . 8
Table X — F1 per workload . . . 8
Table XI — Compute cost . . . 8
Table XII — Predicting deadlock (instead of unsafe) . . . 9
Table XIII — How unsafe episodes ended . . . 9
Table XIV — Robustness to wrong claims . . . 9
Table XV — Changing the warning horizon . . . 9
Table XVI — Speed of the Banker's safety check . . . 10

6. Every figure explained **10**

Figure 1 — Precision vs recall 10
Figure 2 — Accuracy vs compute cost 11
Figure 3 — Precision vs warning time 11
Figure 4 — How the safety check's time grows with system size 13

7. Open anomalies (things we cannot fully explain yet) **13**

8. Limitations (what we did *not* do) **13**

9. Glossary **13**

1. The paper in one minute

Operating systems hand out resources (memory blocks, printers, GPUs, locks) to processes. If they hand them out carelessly, processes can end up stuck forever, each holding something another one needs. That is a **deadlock**.

The classic textbook defence is **Banker's algorithm**. Before granting a request it asks: *"If I grant this, can everyone still finish in some order?"* If yes, the state is **safe**. If no, it is **unsafe**. But Banker's algorithm only answers **"are we safe right now?"** It never says **"we are heading towards danger."**

Our paper asks a new question: **can we warn *before* the system becomes unsafe — how early, how accurately, and at what cost?** Think of it as a smoke detector that beeps before the fire alarm.

What we found, in plain words:

- A **cheap rule-based score** (our method) warns about **10 steps early**. It catches about a third of the dangerous moments, and about 1 in 4 of its alarms is correct. That is far better than guessing (random alarms) and better than simply watching free resources. It is **not** the most accurate method.
- A **machine-learning model** fed the same cheap signals does clearly better (F1 0.48 vs 0.28).
- Methods that **run Banker's algorithm a few extra times** are the most accurate (F1 0.62–0.75), but they **warn later** (about 4 steps ahead) and **cost more compute**.
- A **safety margin** (7 Banker checks per step) comes within **0.03** of the most accurate method (exact lookahead), which for 200 processes costs about **43 times** a normal step.
- **Best practical design:** run the cheap score all the time, and run the costlier safety check only when the cheap score raises an alarm.

Everything was tested in a **simulator**, not on a real operating system. That is the main limitation.

2. Background you need

2.1 Resources, claims and needs

Imagine a bank with a fixed amount of cash (the **resources**). Each customer (a **process**) declares up front the **maximum loan** they might ever need (the **maximum claim**). At any moment:

- **Allocation**: what each customer already has.
- **Need**: maximum claim minus allocation, i.e. how much more they might still ask for.
- **Available**: the cash still in the bank.

A system has n processes and m resource types (for example 200 processes and 20 resource types).

2.2 Safe, unsafe and deadlocked

- **Safe**: there is *some* order in which every customer can get their full need, finish, and return everything. Nobody can get stuck.
- **Unsafe**: no such order is guaranteed. Getting stuck is *possible*, but not certain, because some customers may never ask for their full maximum.
- **Deadlocked**: some processes are actually stuck right now, each waiting for something another holds.

Key fact: **a deadlocked state is always unsafe, but an unsafe state is not always deadlocked.** Unsafe is the "danger zone"; deadlock is the "crash". Our paper's main target is predicting entry into the danger zone (**unsafe onset**), and its second target is predicting the crash (**deadlock onset**).

2.3 Why "observer mode"?

If we let Banker's algorithm refuse dangerous requests, the system would never become unsafe and there would be nothing to predict. So our simulator **grants every valid request** and runs Banker's algorithm only to **watch** and record when the state becomes unsafe. This is called **observer mode**.

3. What we built

3.1 The simulator

At every **step**:

1. Waiting processes are retried (first come, first served).
2. One process makes a new request. It is granted if enough is free; otherwise the process waits.
3. Finished processes give everything back.
4. (Normal workload only) a process may return part of its resources early.
5. We record: is the state unsafe? Is it deadlocked? What are the warning signals?

A run of the simulator is called a **sequence**. It ends at the first deadlock, when all processes finish, or at a step limit.

We checked the simulator with **134 automated tests**. For example, our fast Banker's check gives the same answer as the textbook version on 2,000 random states.

3.2 Three workloads (kinds of behaviour)

Workload	What happens	Real-life analogy
Normal	Small requests, sometimes early releases. Danger builds slowly.	A steady office server
Spike	Normal at first, then suddenly 80% of processes grab big chunks.	Many jobs starting at once on a cluster
Deadlock-prone	Process *i* prefers resource *i+1*, which creates circular waiting.	The classic "dining philosophers" pattern

Each workload was run at **5 sizes**: 10, 25, 50, 100 and 200 processes.

3.3 The five warning signals (indicators)

Each is a number between 0 and 1, computed every step at almost no cost:

Indicator	Meaning in simple words
Scarcity	How much of the total resources is already handed out.
Need pressure	How large the outstanding needs are compared with what is free.
Contention	For how many resource types at least two processes need more than is free (they will fight).
Waiting ratio	What fraction of processes are blocked, waiting.
Request rate	Whether requests are suddenly speeding up.

3.4 Our method: the rule-based score

We combine the five indicators into one **risk score from 0 to 100** (a weighted average). An **alarm** goes off when the score stays above a **threshold** for k steps in a row. After an alarm there is a short quiet period (R steps) so it does not beep continuously. There is no machine learning, so every alarm can be explained ("contention was high").

3.5 The twelve methods we compared

Method	Idea	Cost
Random	Alarms at random, at the same rate as ours (the "guessing" baseline).	free
Availability	Alarm when free resources run low (the obvious simple rule).	free
4 single indicators	Use just one indicator alone.	free
Rule-based (ours)	Weighted average of all five indicators.	free
Rule-based, max	Use the most alarming indicator instead of the average.	free
Logistic regression	Simple ML model on the same signals.	free
Boosted trees	Stronger ML model (gradient-boosted decision trees) on the same signals.	free
Safety margin	"How much free resource could disappear before we become unsafe?" Needs 7 Banker checks.	moderate
Top-3 lookahead	"If the 3 biggest requests were granted, would we become unsafe?" Up to 3 Banker checks.	moderate
Exact lookahead	Same question for **every** running process. One Banker check per process.	high (grows with n)

"Free" means almost no extra compute. The last three are **state-based checks**: they run Banker's algorithm on hypothetical futures.

3.6 How we judged a warning

We borrowed this from hospital early-warning systems (which predict patient deterioration) and earthquake early warning.

- An alarm is **correct (true)** if the system becomes unsafe within the next **20 steps** (the **horizon**). Otherwise it is a **false alarm**.
- An unsafe onset is **caught** if a correct alarm came before it. An alarm *at* the onset step does not count; that is detection, not prediction.
- **Precision** = correct alarms ÷ all alarms. *"When it beeps, how often is it right?"*
- **Recall** = caught onsets ÷ all onsets. *"Of all dangers, how many did it warn about?"*
- **F1** = one number that balances precision and recall (0 = useless, 1 = perfect). It is the **main score** in the paper.
- **Lead time** = how many steps before the onset the alarm came. *"How much warning did we get?"*
- **False alarms per 1,000 steps** = how annoying the method is.
- **NAB score** = a score from 0 (silent) to 100 (perfect, earliest possible warning) that rewards early warnings.

3.7 Fair testing

- **Tuning vs test.** Each method's settings (threshold, k, R, weights) were chosen on **tuning sequences** (45,000), and all reported numbers come from **separate test sequences** (150,000). This is like practising on one exam paper and being graded on a different one.
- **Tuning rule**: maximise F1, with at most 5 false alarms per 1,000 steps and at least 3 steps of median warning.
- **Scale**: 195,000 sequences and 440 million simulation steps on one AWS EC2 machine with 32 CPU cores, taking 5 h 40 min and about $8.39.

3.8 Two pieces of statistics, explained

Confidence interval (CI). "F1 0.280 [0.277, 0.283]" means we are 95% confident the true value lies between 0.277 and 0.283. It is narrow because we have so much data.

Independent batches. A reviewer could say: "your intervals are narrow only because all runs come from the same generator." So we split the 150,000 test sequences into **20 independent batches** and checked every comparison **inside each batch**. If method A beats B in **20 out of 20** batches, the result is very solid.

Equivalence test (TOST). To claim two methods are "about equally good" you cannot just say "the numbers look close". You must pick a tolerance in advance (we used ±0.05 F1) and show statistically that the difference lies inside it. That is what TOST (two one-sided tests) does.

4. The three hypotheses and their verdicts

	Hypothesis	Verdict
H1	The rule-based score warns better than random alarms and the free-resource rule.	**Supported**: better in 20/20 batches.
H2	The cheap safety margin is equivalent to the expensive exact lookahead (within ±0.05 F1).	**Supported, but exploratory**: they differ by 0.027 and are equivalent within ±0.05. The ±0.05 was chosen after seeing the data.
H3	Claim-based indicators do not break when declared maximum claims are slightly wrong.	**Supported for random errors** (change ≤ 0.004). Systematic under-claiming was **not tested**.

Honesty note: these hypotheses were written **after** our earlier exploratory runs, not before. The paper says so openly. That is why every result is called "exploratory".

5. Every table explained

Table I — Scope of related work

What it shows. Earlier research, one row per group of papers, with ticks for what each covers: Banker's safety, forecasting the transition, lead time, event-based evaluation, compute cost. P = peer-reviewed paper, A = preprint (not yet peer-reviewed).

How to read it. Look down each column for ticks.

What to conclude.

- Banker's papers (Dijkstra, Habermann, Belik…) cover safety but **never forecasting** or lead time.
- Early-warning papers from medicine and seismology (Hyland, Münchmeyer) cover forecasting and lead time but **not Banker's safety**.
- Only **"This work"** has all five ticks. This is exactly how our work differs from existing work.

The caption says the entries "describe scope, not shortcomings": we are not saying other papers are bad, just that they asked different questions.

Table II — Methods compared

What it shows. The 12 methods, what signal each uses, whether it is trained (ML), and its extra compute per step compared with one simulation step at 200 processes.

What to conclude. Nine methods are essentially **free**. Safety margin (1.66×) and top-3 lookahead (0.91×) are **moderate**. Exact lookahead (43.45×) is **expensive**. Keep these costs in mind when reading the accuracy tables.

Table III — Claim taxonomy ("what we claim and what we don't")

What it shows. For each kind of result, what the paper *does* claim and what it explicitly *does not* claim.

What to conclude. It guards against over-reading. Examples:

- We claim accuracy **in these workloads**, not on real systems.
- We claim the cost of **our NumPy code**, not of all possible implementations.
- We claim that 77–100% of unsafe episodes ended in deadlock **in our simulator**, not in real systems.

If a professor asks "does this work on Linux?", this table is the honest answer.

Table IV — Experimental runs

What it shows. We ran the whole pipeline three times:

1. **Pilot** on a laptop (48,000 sequences): to check everything works.
2. **Exploratory** cloud run (300,000 sequences): fewer methods.
3. **Main** cloud run (195,000 sequences): all 12 methods. **Every number in the paper comes from this run.**

What to conclude. Results are never mixed across runs. The first two runs only helped us form the hypotheses, which is why the paper calls its results exploratory.

Table V — Test set per workload and size

What it shows. For each workload × size (15 "cells"), the number of sequences (10,000 each), steps, unsafe onsets, % of sequences with an onset, % ending in deadlock, and claim tightness.

How to read it. Claim tightness = total declared maximum claims ÷ total resources. 1.43 means processes together *could* claim 43% more than exists, which is what makes danger possible.

What to conclude.

- Every workload produces a healthy mix of dangerous and quiet sequences (e.g. 11% to 93% contain an onset), so methods are tested on both.
- In the **spike** and **deadlock-prone** workloads, every sequence with an onset ended in deadlock. In **normal**, slightly fewer did, because processes can release resources early and recover.
- 150,000 test sequences and 338 million steps make the numbers statistically stable.

Table VI — Main results (the most important table)

What it shows. For every method except exact lookahead, on all 150,000 test sequences: precision, recall, F1 with 95% interval, false alarms per 1,000 steps, median lead time (steps) and NAB score.

How to read it. Higher F1 = better overall. Higher lead = earlier warning. Lower false alarms = less annoying.

What to conclude.

- **Our rule-based score: F1 0.280, median lead 10 steps.** It clearly beats **random** (0.006) and **availability** (0.162). It works.
- **Contention alone (0.338) beats our combination.** Averaging five signals *dilutes* the best one (Table IX explains why). This is an honest negative result.
- **Boosted trees (0.477)** on the same cheap signals do much better. The signals contain more information than a weighted average can use.
- **Safety margin (0.625)** and **top-3 lookahead (0.576)** are the most accurate here, but warn only **4–5 steps** ahead, versus 10 for ours.
- Rule-based, max has exactly the same numbers as contention. That is not a mistake: with the tuned weights (contention counted double), the maximum is always contention.

One-line takeaway. Cheap = early but less precise; Banker-based = precise but late and costlier.

Table VII — Paired differences across 20 batches

What it shows. For pairs of methods, the F1 difference (first minus second) averaged over 20 independent batches, its 95% interval, and in how many batches the first method won. "All" = full test set; "subset" = sequences where exact lookahead was computed.

How to read it. A positive difference means the first method is better. "20/20" means it won every time. "0/20" means it lost every time.

What to conclude.

- Rule-based vs random **+0.274** (20/20) and vs availability **+0.118** (20/20) means **H1 is solidly confirmed**.
- Rule-based loses to contention, logistic regression and boosted trees in **every** batch. This is not a fluke.
- Safety margin vs exact lookahead **–0.027** in every batch. The margin is really slightly worse, but by only 0.027 F1.
- Every interval is narrow and excludes zero, so all rankings are reliable.

Table VIII — All methods on the common subset

Why it exists. Exact lookahead is expensive, so we computed it only on 61,350 sequences (all small systems, plus a sample of large ones). Comparing it with methods measured on all 150,000 sequences would be unfair, like comparing two students graded on different exams. So this table puts **all 12 methods on the same 61,350 sequences**.

What to conclude.

- **Exact lookahead 0.745**, **safety margin 0.718**, **top-3 0.693** are the top three, all warning 4–5 steps ahead.
- Our rule-based score gets **0.443** here, higher than on the full set (0.280), because this subset is dominated by small systems, which are easier (see anomaly A4).
- Logistic regression shows 5.44 false alarms per 1,000 steps here (anomaly A6): its settings do not transfer well to small systems.

Table IX — Ablation (removing one signal at a time)

What it shows. The rule-based score with equal weights and all five signals (F1 0.24), and then with each signal removed in turn (each version re-tuned).

How to read it. If F1 *drops* when a signal is removed, that signal was helping. If F1 *rises*, that signal was hurting.

What to conclude.

- Removing **contention** drops F1 to **0.20**: contention is the most useful signal.
- Removing **waiting ratio** raises F1 to **0.27**: in an average, it adds noise.
- Lesson: **a simple average is not the best way to combine signals.** That is why the tuned version doubles contention's weight, and why boosted trees do better.

Table X — F1 per workload

What it shows. F1 of every method separately for normal, spike and deadlock-prone workloads.

What to conclude.

- **Spike is easiest for cheap signals** (rule-based 0.42 vs 0.22–0.23 on the others). A sudden burst of grabs is easy to notice: onsets came a median 26 steps after the spike started.
- **Gradual danger (normal, deadlock-prone) is hard for cheap signals**, but the Banker-based checks still do well (safety margin 0.55 and 0.54).
- Waiting ratio is good on normal (0.29) and deadlock-prone (0.31) but almost useless on spike (0.03). That is anomaly A1 (Section 7).
- Request rate is about 0 except on spike. It only detects sudden bursts, which makes sense.

Takeaway. Which method to use depends on how danger builds up in your system.

Table XI — Compute cost

What it shows. Microseconds (µs) per step for each costly computation, and the same as a multiple of one simulation step, averaged over all sizes and at 200 processes.

What to conclude.

- One simulation step takes about **289 µs**, and one Banker safety check about **60 µs**.
- Safety margin costs **1.44×** a step on average and **1.66×** at 200 processes. Its cost barely grows because it always uses exactly 7 checks.

- Exact lookahead costs **6.4×** on average but **43.45×** at 200 processes, because it does one check per process, so its cost grows with system size.
- **Important caveat:** these are costs of *our* Python/NumPy code. A faster implementation would change the absolute times. The *ratios* are the lesson, not the microseconds.

Table XII — Predicting deadlock (instead of unsafe)

What it shows. The same methods, now judged on warning before an actual **deadlock**, on the same 25,862 deadlock onsets for all methods. The last column shows the full-population F1 (77,700 onsets) for comparison.

What to conclude.

- On equal ground, **our rule-based score is the best deadlock predictor (F1 0.659)**, followed by availability (0.552) and boosted trees (0.540).
- The Banker-based checks, which were the best at predicting *unsafe* states, are **among the weakest at predicting deadlock** (0.30–0.37). Likely reason: they are only computed while the state is still safe, so they go silent during the unsafe stretch just before the deadlock (anomaly A3).
- **Lesson:** the ranking of methods depends on what exactly you are predicting.

This table was fixed after review. The earlier version compared methods on different sets of sequences and gave a misleading ranking.

Table XIII — How unsafe episodes ended

What it shows. For each workload, how many unsafe episodes recovered (became safe again) and how many ended in deadlock.

What to conclude.

- Spike and deadlock-prone: **100%** ended in deadlock. Normal: **77%** (4,912 recovered).
- **Caution:** this is largely a property of our simulator. Processes always eventually ask for their full maximum claim, and a sequence stops at the first deadlock. In real systems many processes never use their full claim, so far fewer unsafe states would become deadlocks. The paper says this explicitly.

Table XIV — Robustness to wrong claims

What it shows. In real systems, processes may misreport their maximum claims. We made every declared claim randomly wrong by up to ±20% (the true claims were still used to decide what is really unsafe) and re-ran the same tuned methods.

What to conclude.

- F1 changed by at most **0.004**: the methods are robust to **random** errors.
- **Limitation:** random errors cancel out (some too high, some too low). We did **not** test **systematic** errors, such as everyone under-claiming by 20%. That is listed as future work.

Table XV — Changing the warning horizon

What it shows. The same alarms scored with a horizon of 5, 10, 20 or 50 steps. With a 50-step horizon, an alarm counts as correct if an unsafe onset happens within the next 50 steps.

What to conclude.

- Every method improves as the horizon widens. Ours goes from 0.108 (5 steps) to 0.399 (50 steps). Many of our "false" alarms at 20 steps were actually **real but very early** warnings.

- The **ranking of methods does not change**, so our conclusions do not depend on the choice of 20 steps.

Table XVI — Speed of the Banker's safety check

What it shows. Time per safety check for our **vectorised** NumPy version and the **textbook** loop version, for different n and m.

What to conclude.

- Our version is **1.4× to 7.7× faster**, and the gap grows with system size.
- Theory says the check costs up to $m \times n^2$ operations (worst case). In practice our version's time grew much more slowly. Doubling n increased time by only about 20%, because typical states are resolved in a few passes. This is **average-case behaviour of one implementation**, not a new theoretical result.

6. Every figure explained

Figure 1 — Precision vs recall

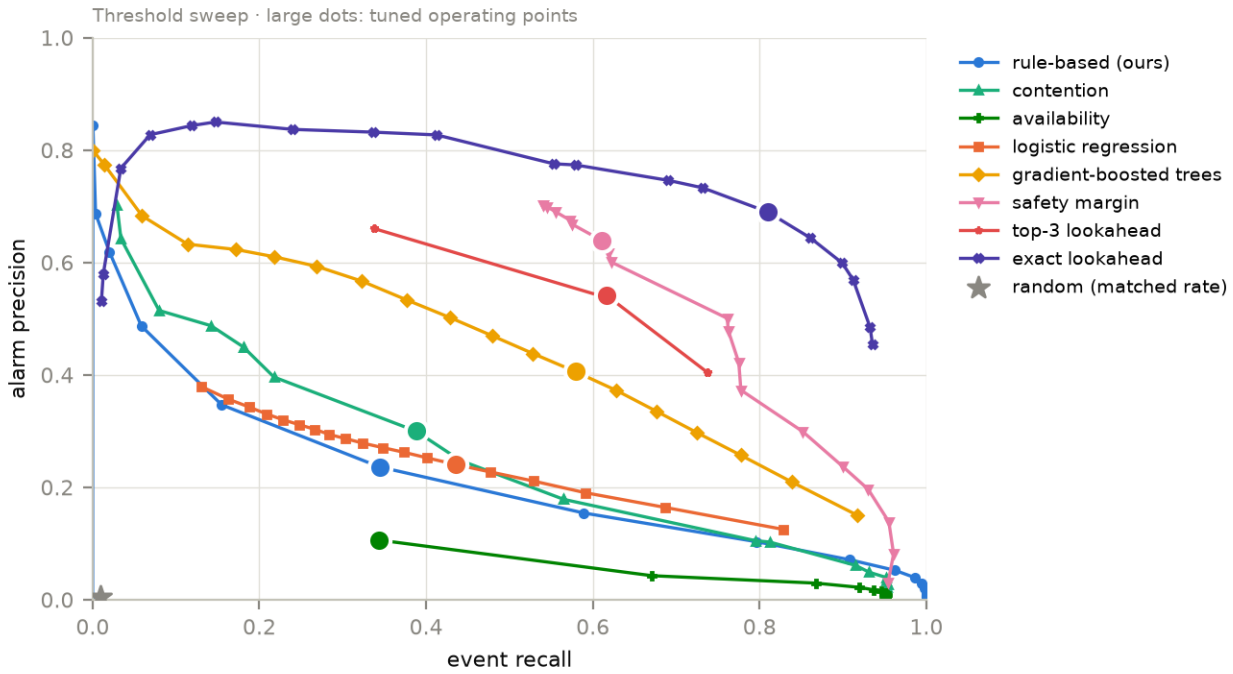


Figure 1: Precision–recall along each threshold sweep

How to read it. Each coloured line is one method, traced by moving its threshold from low to high. A **low threshold** gives many alarms: high recall but low precision (bottom right). A **high threshold** gives few alarms: high precision but low recall (top left). The **big dot** is the setting chosen during tuning. **Further up and to the right is better.** The grey star near (0, 0) is random alarms.

What to conclude.

- **Exact lookahead** (purple) is the highest curve: best at every trade-off point.
- **Safety margin** (pink) and **top-3 lookahead** (red) come next.
- **Boosted trees** (orange) is clearly the best of the cheap methods.
- Our **rule-based** score (blue), **contention** and **logistic regression** are close together. **Availability** (dark green) is low. **Random** is stuck at zero.

- So the ranking in Table VI is not an accident of one threshold: it holds along the whole curve.

Figure 2 — Accuracy vs compute cost

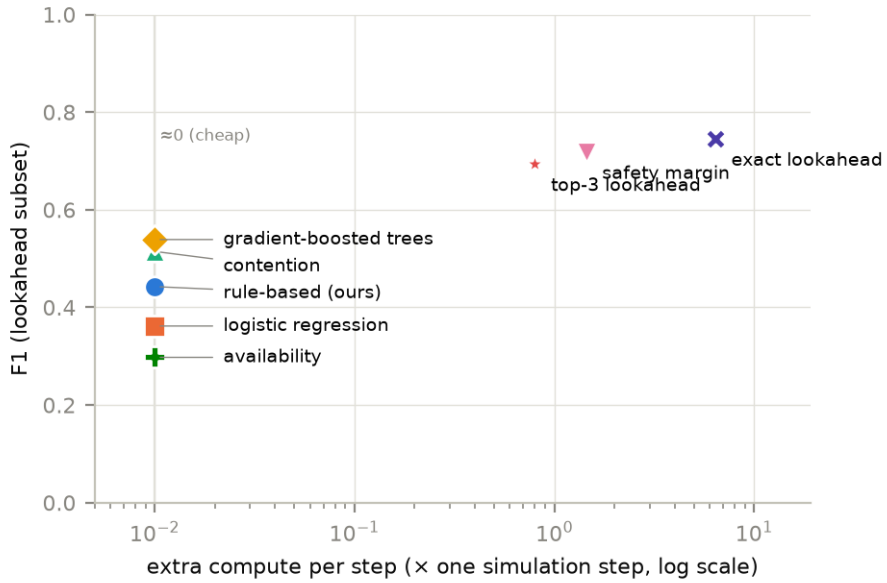


Figure 2: F1 vs extra compute

How to read it. The x-axis shows extra compute per step, as a multiple of one simulation step, on a **log scale** (each grid line is 10× more). The y-axis shows F1 on the common subset. Cheap methods are drawn at the far left ("≈0"). **Top-left is ideal**: accurate and cheap.

What to conclude.

- Moving from the cheap methods (left) to the Banker-based checks (around 1×) gives the **big jump in accuracy** (from about 0.30–0.54 up to 0.69–0.72).
- Going further from the safety margin to exact lookahead costs **several times more** (about 6× here, averaged over sizes; 43× at 200 processes) for only **+0.027 F1**.
- **Practical message:** the safety margin is the "sweet spot" on this frontier.

Figure 3 — Precision vs warning time

How to read it. The x-axis shows how early the warning comes (median lead, in steps); the y-axis shows how often an alarm is right. Each line is again a threshold sweep, and the big dots are the tuned settings. **Top-right is ideal**: early and precise.

What to conclude.

- There is a clear **trade-off**: earlier warnings are less precise for every method.
- The Banker-based methods' big dots sit at **4–5 steps** with precision 0.54–0.69: precise but late.
- The cheap methods' dots sit at **7–12 steps** with lower precision: early but noisier.
- **Boosted trees** (orange, 11 steps, precision 0.41) gives the best early warning among practical methods.
- This is why we suggest **combining** a cheap early score with a precise Banker-based check.

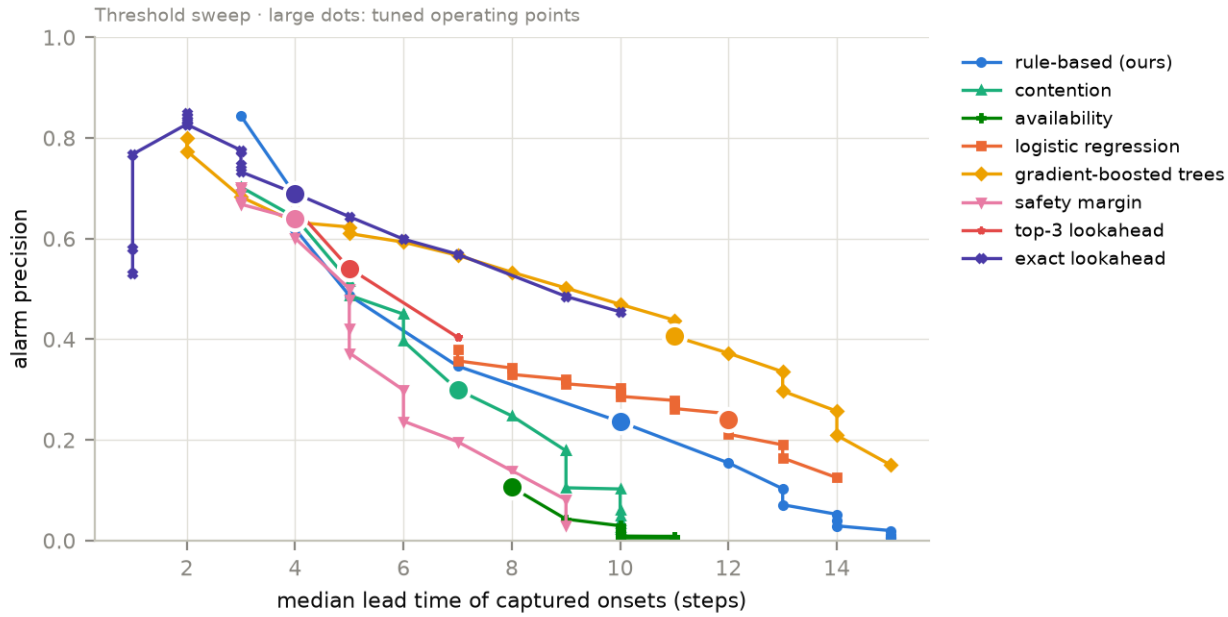


Figure 3: Precision vs median lead time

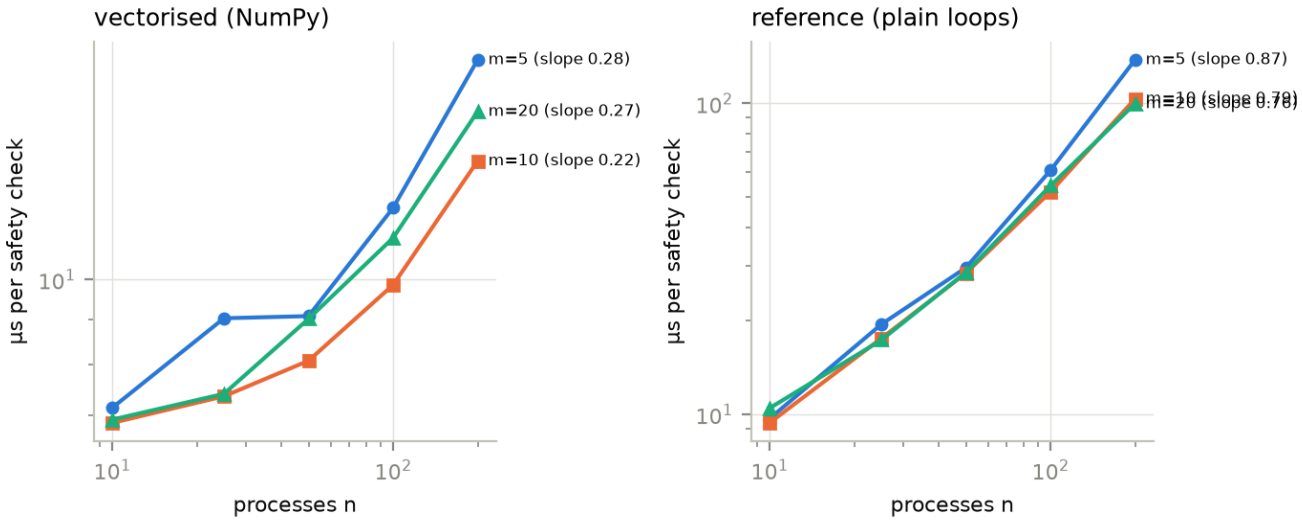


Figure 4: Scaling of the safety check

Figure 4 — How the safety check's time grows with system size

How to read it. Both axes are **log scale**, so a straight line means time grows like n to the power "slope". The **slope** tells how fast the time grows: slope 1 means doubling n doubles the time, and slope 2 would mean 4× the time. Left panel: our vectorised version. Right panel: textbook loops.

What to conclude.

- Our vectorised version has slopes of about **0.22–0.28**: time grows very slowly with n.
- The textbook version has slopes of about **0.76–0.87**: close to linear.
- Both are far below the theoretical worst case (n^2, slope 2), because typical states are easy.

7. Open anomalies (things we cannot fully explain yet)

Good papers admit what they do not understand. Ours lists six puzzles:

	What we saw	Our best guess
A1	Waiting ratio works on normal and deadlock-prone (F1 0.29–0.31) but fails on spike (0.03).	In a spike, grabs are capped by free resources so they are *granted*; few processes are waiting before danger.
A2	Boosted trees catch 98.5% of deadlocks. No other method comes close.	It may simply keep alarming during the unsafe period just before a deadlock.
A3	Banker-based checks are best for unsafe states but weak for deadlocks.	They are computed only while safe, so they go silent during the unsafe stretch that precedes deadlock.
A4	Most methods get worse in bigger systems, but waiting ratio gets better.	Settings were tuned once for all sizes; per-size tuning would test this.
A5	Some single signals chose the highest threshold allowed (95).	Their best setting may lie beyond our search range.
A6	Logistic regression gives many false alarms on small systems (5.44 per 1,000 steps).	Settings tuned on all sizes do not transfer to small systems.

8. Limitations (what we did *not* do)

1. **No real system data.** Everything is simulated. Public cluster traces (Google, Alibaba) do not record maximum claims, so Banker's "unsafe" cannot be computed from them.
2. **No systematic claim errors** (everyone under-claiming).
3. Batches reuse settings that were **tuned once**.
4. The suggested **two-step design** (cheap score, then safety check) was not tested end to end.
5. Costs come from **one implementation** (Python/NumPy).
6. Nothing was **pre-registered**. Hypotheses were written after early runs, so the results are exploratory.

9. Glossary

Term	Simple meaning
Process	A running program asking for resources.
Resource type	A kind of resource (e.g. GPU, printer, memory block).
Maximum claim	The most a process says it will ever need.
Safe state	Everyone can still finish in some order.
Unsafe state	Finishing for everyone is no longer guaranteed.
Deadlock	Processes stuck waiting for each other forever.
Onset	The step at which the system *enters* an unsafe (or deadlocked) state.
Episode	One continuous stretch of being unsafe.
Sequence	One full simulation run.
Step	One tick of the simulator (one new request plus updates).
Alarm	Our warning that danger is coming.
Horizon	How far ahead an alarm may come and still count as correct (20 steps).
Lead time	How many steps of warning an alarm gave.
Precision	Of all alarms, the fraction that were right.
Recall	Of all dangers, the fraction we warned about.
F1	A single score combining precision and recall (0 to 1).
Threshold	The risk-score level above which we raise an alarm.
Tuning / test split	Practice data for choosing settings / separate data for grading.
Confidence interval	The range in which the true value lies with 95% confidence.
Batch	A group of 7,500 test sequences used to check that results repeat.
TOST	A statistical test for "these two are equivalent within a tolerance".
Lookahead	Trying out hypothetical future grants with Banker's algorithm.
Safety margin	How much free resource could vanish before the state turns unsafe.
Vectorised	Written to process many numbers at once (fast NumPy code).
Ablation	Removing one part to see how much it mattered.
Observer mode	Grant everything, and only *watch* with Banker's algorithm.